

2143 Object-Oriented Programming

Exam 03 — Spring 2026 | 80 minutes | 100 points

Instructions:

- Record all answers on the **Answer Sheet**. Nothing written on this booklet will be graded.
- Circle the single best answer for every question in all three parts.
- Assume `using namespace std;` and necessary headers included unless stated otherwise.
- No electronic devices.

1 Multiple Choice (2 pts each — circle the single best answer on the Answer Sheet)

- (2 pts) Runtime polymorphism in C++ requires which **two** conditions to both be true?
 - A. The function is declared `static` in the base class.
 - B. The function is declared virtual in the base class and the call is made through a base class pointer or reference.**
 - C. The derived class uses the `override` keyword.
 - D. The base class must be abstract.
- (2 pts) What is **object slicing**?
 - A. Removing unused methods from a class at compile time.
 - B. When a derived object is assigned to a base object by value, losing derived-specific data and behavior.**
 - C. A compiler optimization that trims the vtable.
 - D. Splitting a large class into multiple smaller ones.
- (2 pts) Given the code below, which version of `greet()` executes?

```
Base* p = new Derived();    // greet() is virtual and overridden in Derived
p->greet();
```

 - A. `Base::greet()`
 - B. `Derived::greet()`**
 - C. Both run in sequence, base first.
 - D. Undefined behavior — must cast to `Derived*` first.
- (2 pts) Which of the following call sites uses **static** (compile-time) dispatch?
 - A. `Base* p = new Derived(); p->foo();`
 - B. `void f(Base& b) { b.foo(); }`
 - C. `Derived d; d.foo();`**
 - D. `Base& r = d; r.foo();`
- (2 pts) `delete a;` is called where `Animal* a = new Dog();`. The destructor `~Animal()` is **not** virtual. What happens?
 - A. Both destructors run in the correct order.

- B. Only `~Dog()` runs.
 - C. Only `~Animal()` runs — derived resources may leak.**
 - D. A compile-time error occurs.
6. (2 pts) What makes a C++ class **abstract**?
- A. Declaring it with the **abstract** keyword.
 - B. Having at least one pure virtual function (`= 0`).**
 - C. Making all data members **private**.
 - D. Inheriting from another class.
7. (2 pts) Which is a correctly declared **pure virtual** function?
- A. `virtual void draw() const;`
 - B. `void draw() const override;`
 - C. `virtual void draw() const = 0;`**
 - D. `static virtual void draw() = 0;`
8. (2 pts) `Shape` declares `virtual double area() const = 0;`. `Circle` inherits `Shape` but does **not** override `area()`. What is true?
- A. `Circle` inherits a default `area()` returning 0.
 - B. `Circle` is also abstract and cannot be instantiated.**
 - C. The program crashes at runtime when `area()` is called on a `Circle`.
 - D. `Circle::area()` returns garbage.
9. (2 pts) Given abstract `Animal` (pure virtual `speak()`) and `Dog` (overrides `speak()`), which line is **legal**?
- A. `Animal a;`
 - B. `Animal* p = new Animal();`
 - C. `Animal* p = new Dog();`**
 - D. `Animal a = Dog();`
10. (2 pts) A class intended to serve as a base class should **always** include:
- A. A static factory method.
 - B. A **protected** default constructor.
 - C. A virtual destructor.**
 - D. At least two pure virtual functions.
11. (2 pts) Which statement about **static member functions** is true?
- A. They have access to the **this** pointer.
 - B. They can only be called through an object instance.
 - C. They cannot directly access non-static data members.**
 - D. Every class must define at least one.
12. (2 pts) A static data member is declared inside the class body. Where must it also be **defined**?

- A. Directly after the declaration, inside the class body.
 - B. In every `.cpp` file that uses the class.
 - C. Outside the class body exactly once — e.g., `int Widget::count = 0;`**
 - D. In the class constructor.
13. (2 pts) `Widget::getCount()` is called without creating any `Widget` object. This is valid because `getCount()` is declared:
- A. `friend`
 - B. `virtual`
 - C. `static`**
 - D. `const`
14. (2 pts) Which relationship is best modeled with **composition** rather than inheritance?
- A. A `Square` and a `Shape`
 - B. A `Dog` and an `Animal`
 - C. A `Car` and an `Engine`**
 - D. A `SavingsAccount` and an `Account`
15. (2 pts) The design principle “favor composition over inheritance” is recommended because composition:
- A. Is always faster at runtime.
 - B. Is more flexible — parts can be swapped and class coupling is reduced.**
 - C. Automatically prevents the diamond problem.
 - D. Always produces shorter code.
16. (2 pts) Which phrase describes an **inheritance** (“is-a”) relationship?
- A. A `House` has a `Room`.
 - B. A `Computer` has a `CPU`.
 - C. A `SportsCar` is a `Car`.**
 - D. A `Library` contains `Book` objects.
17. (2 pts) A `Library` object holds a collection of `Book` objects. This design is an example of:
- A. Inheritance
 - B. Composition**
 - C. Polymorphism
 - D. Abstraction
18. (2 pts) The `protected` access specifier allows a member to be accessed from:
- A. Only the class itself.
 - B. The class and its derived classes, but not external code.**
 - C. All code, including external functions.
 - D. Only `friend` classes and functions.
19. (2 pts) Inside a derived class, which members of the **base** class can be accessed directly?

- A. Only public members.
 - B. public, protected, and private members.
 - C. public and protected members only.**
 - D. Only protected members.
20. (2 pts) A virtual destructor is **required** when:
- A. Any class contains dynamic memory.
 - B. A class has more than three data members.
 - C. An object may be deleted through a base class pointer.**
 - D. A class declares any virtual function.
21. (2 pts) In the hierarchy $A \rightarrow B \rightarrow C$ (A is root), a C object is created then goes out of scope. What is the correct order of calls?
- A. C ctor, B ctor, A ctor, C dtor, B dtor, A dtor
 - B. A ctor, B ctor, C ctor, A dtor, B dtor, C dtor
 - C. A ctor, B ctor, C ctor, C dtor, B dtor, A dtor**
 - D. C ctor, B ctor, A ctor, A dtor, B dtor, C dtor
22. (2 pts) The primary purpose of the **override** keyword (C++11) is to:
- A. Force the compiler to inline the function.
 - B. Make the function virtual automatically.
 - C. Have the compiler verify the function actually overrides a base virtual — catching typos and signature mismatches.**
 - D. Prevent the function from being overridden in deeper subclasses.
23. (2 pts) The **diamond problem** in C++ is resolved using:
- A. The **abstract** keyword
 - B. The **final** keyword
 - C. virtual inheritance**
 - D. Multiple **override** declarations
24. (2 pts) What happens to an exception that is **never caught** anywhere in the program?
- A. It is silently ignored.
 - B. It becomes a compiler warning on the next build.
 - C. `std::terminate()` is called, typically ending the program immediately.**
 - D. It is returned as the exit code of `main`.
25. (2 pts) Which smart pointer enforces **exclusive** (sole) ownership and **cannot** be copied?
- A. `std::shared_ptr` (B) `std::weak_ptr`
 - B. `std::unique_ptr` (D) `std::auto_ptr`**
-

2 Matching (1 pt each — write the letter on the Answer Sheet)

Terms (left)

Descriptions (right)

- | | |
|--------------------------|--|
| 1. <code>virtual</code> | |
| 2. <code>override</code> | |
| 3. <code>final</code> | |
| 4. <code>= 0</code> | |
| 5. <code>friend</code> | |
| 6. <code>throw</code> | A. Compiler-enforced check that a method actually overrides a base class virtual. |
| 7. <code>catch</code> | B. Signals that an exceptional situation has occurred; propagates up the call stack. |
| 8. <code>template</code> | C. Begins a generic, type-parameterized function or class definition. |
| 9. <code>RAII</code> | D. Makes a virtual function pure — the containing class becomes abstract. |
| 10. Object Slicing | E. Prevents a virtual function from being further overridden, or a class from being inherited. |
| | F. Grants an outside function or class direct access to <code>private</code> members. |
| | G. Handles a thrown exception; matched by type. |
| | H. Ties a resource's lifetime to a stack object's lifetime via constructors and destructors. |
| | I. Marks a method as participating in runtime (dynamic) dispatch. |
| | J. Loss of derived-class data when a derived object is copied to a base variable by value. |

Solution: Matching Key:

- | | | | | |
|-------------------------|--------------------------|--------------------------|----------------------|---------------------------------|
| 1. I (<i>virtual</i>) | 2. A (<i>override</i>) | 3. E (<i>final</i>) | 4. D (<i>= 0</i>) | 5. F (<i>friend</i>) |
| 6. B (<i>throw</i>) | 7. G (<i>catch</i>) | 8. C (<i>template</i>) | 9. H (<i>RAII</i>) | 10. J (<i>Object Slicing</i>) |

3 Code Analysis (2.5 pts each — circle the single best answer on the Answer Sheet)

Read each code block carefully, then answer the four questions that follow it.

Questions 26–29 refer to the following program.

```
1 class Base {  
2 public:  
3     virtual void greet() { cout << "Hello from Base\n"; }  
4     virtual ~Base()      { cout << "~Base\n"; }  
5 };  
6 class Derived : public Base {  
7 public:  
8     void greet() override { cout << "Hello from Derived\n"; }  
9     ~Derived()           { cout << "~Derived\n"; }  
10 };  
11 void test1(Base b) { b.greet(); }    // by value  
12 void test2(Base& b) { b.greet(); }  // by reference  
13  
14 int main() {  
15     Derived d;  
16     test1(d);  
17     test2(d);  
18     Base* p = new Derived();  
19     p->greet();  
20     delete p;  
21 }
```

26. (2 pts) What does `test1(d)` print?

- A. Hello from Derived
- B. Hello from Base
- C. Nothing — the copy constructor suppresses output.
- D. Compile error — cannot pass `Derived` where `Base` is expected.

27. (2 pts) What does `test2(d)` print?

- A. Hello from Base
- B. Hello from Derived
- C. Nothing — references do not invoke virtual dispatch.
- D. Undefined behavior.

28. (2 pts) After `delete p;` executes, which two lines of output appear **in order**?

- A. `~Base` then `~Derived`
- B. `~Derived` then `~Base`
- C. Only `~Base`
- D. Only `~Derived`

29. (2 pts) What concept explains why `test1` and `test2` produce *different* output even though `greet()` is virtual?

- A. The `override` keyword is missing on `Derived::greet`.

- B. Object slicing — passing by value strips the derived part, preventing virtual dispatch.
- C. virtual dispatch requires pointers; it does not work with references.
- D. Base is abstract and cannot be passed by value.

Questions 30–33 refer to the following program.

```
1 class Animal {
2 public:
3     Animal() { cout << "Animal ctor\n"; }
4     ~Animal() { cout << "Animal dtor\n"; } // <-- examine this
5 };
6 class Dog : public Animal {
7     int* tricks;
8 public:
9     Dog() { tricks = new int[50]; cout << "Dog ctor\n"; }
10    ~Dog() { delete[] tricks; cout << "Dog dtor\n"; }
11 };
12 int main() {
13     Animal* a = new Dog();
14     delete a;
15 }
```

30. (2 pts) What is the defect in this program?
- A. Dog does not explicitly call the Animal constructor.
 - B. **~Animal() is not virtual, so ~Dog() never runs when deleting through Animal*.**
 - C. tricks is not initialized to nullptr before assignment.
 - D. delete[] should be delete.
31. (2 pts) What is the runtime consequence of this defect?
- A. Compile error — the program never builds.
 - B. The program crashes immediately on the delete a line.
 - C. **~Dog() is skipped and the tricks array is leaked.**
 - D. Both destructors run but in the wrong order.
32. (2 pts) What is the **minimum** fix?
- A. Add virtual to ~Dog().
 - B. **Change ~Animal() to virtual ~Animal().**
 - C. Replace delete a with delete (Dog*)a.
 - D. Add override to ~Dog().
33. (2 pts) After applying the fix, what is the **complete** output of main()?
- A. Animal ctor → Dog ctor → Animal dtor
 - B. **Animal ctor → Dog ctor → Dog dtor → Animal dtor**
 - C. Dog ctor → Animal ctor → Dog dtor → Animal dtor
 - D. Animal ctor → Dog ctor → Dog dtor

Questions 34–37: A game requires *Player* objects. Each *Player* has-a *Weapon*. The engine must track the total number of active *Player* objects without iterating a list.

34. (2 pts) Which implementation correctly models the *has-a* relationship?

- A. `class Player : public Weapon { ...};`
- B. `class Weapon : public Player { ...};`
- C. `class Player { Weapon weapon; ...};`
- D. `class Player { Weapon* weapon = nullptr; ...};` (raw pointer only valid option)

35. (2 pts) Where must the line `int Player::count` be written?

- A. Inside the class body, immediately after `static int count;`.
- B. In the `Player` constructor, as `count = 0;`.
- C. **Outside the class body exactly once — e.g., `int Player::count = 0;` in a `.cpp` file.**
- D. Nowhere — the declaration inside the class is sufficient.

36. (2 pts) Three `Player` objects are created, then one goes out of scope. What does `Player::getCount()` return?

- A. 3 (B) 2 (C) 1 (D) 0

37. (2 pts) Why can `Player::getCount()` be called without any `Player` object in scope?

- A. It is declared `const`.
- B. It is declared `virtual`.
- C. **It is declared static — static methods belong to the class, not to any instance.**
- D. It is declared `inline`.

Questions 38–41 refer to the following two programs.

```

1 // Program A
2 template <typename T>
3 T bigger(T a, T b) { return (a > b) ? a : b; }
4
5 int main() {
6     cout << bigger(3, 7) << "\n";
7     cout << bigger(3.14, 2.71) << "\n";
8     cout << bigger(string("apple"), string("banana")) << "\n";
9 }

```

```

1 // Program B
2 class Shape {
3 public:
4     double area() const { return 0; }    // <-- is this right?
5     ~Shape() {}
6     void describe() const { cout << "Area: " << area() << "\n"; }
7 };
8 class Rectangle : public Shape {
9     double width, height;
10 public:
11     Rectangle(double w, double h) : width(w), height(h) {}
12     double area() const { return width * height; }
13 };

```

38. (2 pts) Program A: what does `bigger(string("apple"), string("banana"))` return?
- A. "apple" — alphabetically first.
 - B. "banana" — lexicographically greater, so `operator>` returns true.
 - C. Compile error — `operator>` is not defined for `string`.
 - D. Undefined — string comparison is not portable across platforms.
39. (2 pts) Program A: `bigger(3, 4.5)` causes a compile error. Why?
- A. Templates cannot mix numeric types under any circumstances.
 - B. Template argument deduction fails — `T` cannot be both `int` and `double` simultaneously.
 - C. `operator>` is ambiguous for mixed types.
 - D. `4.5` is not a valid template argument.
40. (2 pts) Program B: `Shape* s = new Rectangle(3, 4); s->describe();` prints `Area: 0`. Why?
- A. `Rectangle::area()` has a bug — it should return `width + height`.
 - B. `describe()` cannot call `area()` through a pointer.
 - C. `Shape::area()` is not virtual, so `describe()` calls `Shape::area()` statically, ignoring `Rectangle`'s override.
 - D. `Rectangle` must be cast to `Shape` before `area()` resolves correctly.
41. (2 pts) Which change to Program B's `Shape` class **both** fixes the output **and** prevents `Shape` from being instantiated directly?

- A. Change `~Shape()` to `virtual ~Shape()`.
- B. Add `override` to `Shape::area()`.
- C. **Change `area()` to `virtual double area() const = 0;` and add `virtual ~Shape()` `{}`.**
- D. Add `final` to the `Shape` class declaration.